INVOICE 001 · PLATE-SOLVER V1 · 25 JUN — 01 JUL 2026

WE BUILT IT FOR $61.

Tokenomics is not the price of tokens. It is **which model does which job,** and **how many times you pay to re-read the same context.** Here is one week of it, itemised.

QTY	DESCRIPTION	AMOUNT
320 M	**TOKENS DELIVERED** at frontier list, uncached	$1,695.80
	LESS: PROMPT CACHE REUSE the same context, not bought twice	−$1,121.97
6,541	**LESS: WORK ROUTED OFF THE METER** local model and flat-fee tiers	−$512.70
	TOTAL DUE	**$61.13**

PAID

Nobody gave us a discount. Every credit on this bill is an architecture decision.

THE WHOLE TALK, UP FRONT

TWO LEVERS. NEITHER ONE IS A DISCOUNT.

01 THE RIGHT MODEL FOR THE JOB

Writing a function against a written spec is not the same task as deciding whether that function is correct. We price them the same anyway — everything goes to the best model we have, out of habit.

Sort the work first. **Judgment goes to the frontier model. Volume goes to a small model on your own hardware.** The router is the product decision; the model is not.

NEVER LEFT THE LAPTOP

of every call in the build

85%

02 PAY ONCE FOR THE SAME CONTEXT

An agent re-reads its whole world on *every single turn* — the spec, the code, the conversation so far. Priced fresh each time, that repetition is most of what appears on the invoice.

Cached, those identical tokens cost **a tenth.** It is the largest lever in this deck and the least glamorous: no vendor call, no model change. **Ask your team their cache hit rate.** Most cannot answer.

SAVED BY CACHING

on one week of this build

$1,122

Everything after this slide is **evidence for those two lines** — a real build, instrumented, with the receipts.

THE VOCABULARY · WHAT THE WORD ACTUALLY MEANS

ONE TOKEN. FOUR WAYS TO LOOK AT IT.

The same atom, priced by four different people. Which face you are looking at decides whether this is a procurement conversation or an engineering one.

01

COGNITION

What the model produces. One unit of thinking — a word, a step, a decision.

02

COMPUTE

What the data center serves. Silicon, power, and capacity someone had to build.

03

PRICE

What the lab charges. The number on the rate card everyone quotes.

THE ONLY FACE WITH A PRICE LIST

04

VALUE

What your business extracts. Work finished, to an acceptable standard.

THE ONLY FACE ANYONE IS PAID ON

The lab bills you on **price**. You are paid on **value**. Tokenomics is the practice of managing the distance between those two — and nearly all of that distance is architecture.

Frame: Slalom FinOps for AI · Tokenomics POV.

WHAT DRIVES THE BILL

FOUR DECISIONS, ALL MADE BEFORE FINANCE SEES ANYTHING.

None of these is a price. Every one of them is a choice someone made in a design review, and every one of them compounds.

	THE DECISION	WHAT THIS BUILD DID ABOUT IT
01	**FRONTIER BY DEFAULT** the best model on hand, before anyone defined “good enough”	Sorted the work first. Judgment out, volume in-house. ROUTING
02	**GROWING CONTEXT** prompts, history, retrieval and tools, re-read on every turn	Paid once for it. Fresh agent per task, so it never compounds. CACHING
03	**LOOPS AND RETRIES** one request becomes a hundred calls, and failures still bill	An independent judge, a hard retry cap, then it parks for a human. VERIFY & CAP
04	**NO NAMED OWNER** finance sees the bill after the architecture is already embedded	One board, per project, per model, read every morning. ATTRIBUTION

Three of the four are architecture. **The fourth is why the other three never get fixed.** The rest of this talk is one week of doing something about each of them.

Frame: Slalom FinOps for AI · Tokenomics POV.

A PHOTO OF THE SKY IN.

Real output. No GPS, no timestamp, no metadata — just pixels.

READOUT	VALUE
SOLVE TIME	1.8 ms
STARS MATCHED	47
PATTERN DATABASE	1.01 M

Astronomy software that works out where a telescope was aimed — from the pixels alone. It is how a satellite finds itself when it is lost in space.

It is real, and it is benchmarked. Rust, gRPC service, web UI, runs on Raspberry-Pi-class hardware, and it passes a parity test against an independent Python reference it did not write.

That matters for one reason: the bill I am about to show you bought working software, not a demo. The product is the receipt — not the subject.

HOW THE WORK GOT DONE

NOTHING IN THE LOOP COSTS MONEY.

PLAN.MD

Written once, by a human. The spec, the task list, and the definition of done — the only durable state, which is why the loop survives a restart.

THE ONLY METERED THING

$60.80

Me, on a frontier model, **watching it run.** 5.5% of the calls — and 99.5% of the bill.

SCHEDULE A · REPEATS PER TASK

01 **PICK THE NEXT TASK** — fresh agent, empty context — $0

02 **LOCAL MODEL WRITES IT** — qwen3.6-27b · on this Mac — $0

03 **JUDGE REVIEWS IT** — glm-5.2 · a different model family — $0

↻ FAIL → back to 02 · retry ≤ 3×, then park for a human

04 **GATES RUN, THEN COMMIT** — compiler · tests · parity oracle — $0

↻ Next task — agent exits, context resets, cost stays flat

SAME PROJECT · SAME SPECS

ONE WEEK BY HAND. ONE WEEK ON THE LOOP.

TOKENS DELIVERED

+65% more work

TOTAL PAID

−85% spend

COST PER MTOK

11× cheaper

Honest but not perfect. The manual week included spec authoring and architecture — different work, not just slower coding. The unit-cost trend is the claim; a task-for-task comparison would be tighter.

Against the counterfactual: those same 320 million tokens run entirely on the frontier model would have cost **$574**. Without caching on top of that, **$1,696**.

LEVER ONE

85% NEVER LEFT THE BUILDING.

Here is where every call in that week actually went — and what each tier cost.

Note the shape. **Volume and spend point in opposite directions.** The tier doing almost all the work is the tier that bills nothing; the tier doing 5.5% of the calls is essentially the entire invoice.

That is what routing buys you, and it is why the interesting question is never *“which model is best”* — it is **which of these rows does this particular task belong in.**

And it was a config file, not a rewrite. Claude Code asks for a model *by name*; the switchboard decides what actually answers and the tool never finds out. Ask for “sonnet”, get the local model. Same commands, same repo, nothing to procure — the local tier is a laptop that was already on the desk.

WHERE IT RAN	CALLS	COST
LOCAL on this Mac · writes the code	5,880	ELECTRICITY
FLAT-FEE CLOUD the judge · already paid for	654	$0 MARGINAL
FRONTIER human oversight · 5.5% of calls	381	$60.80

At frontier rates the judge alone would have been **$160** — taking the build from $61 to $221.

LEVER TWO · THE BIGGEST NUMBER HERE

MOST OF WHAT YOU PAY FOR IS RE-READING.

Lever two, and the largest number in this deck — larger than every routing decision combined.

Read the three rows as one sentence. Run the obvious way, this build was **$1,696**. Turning on caching alone took **$1,122** off it. Routing then took most of the rest.

Which is the uncomfortable part: **the biggest lever required no model change at all.** It is a header, a stable prompt prefix, and someone bothering to check the hit rate. Pure engineering, and nobody gets promoted for it.

RUN THE OBVIOUS WAY

$1,696

No caching, no routing

SAVED BY CACHING ALONE

−$1,122

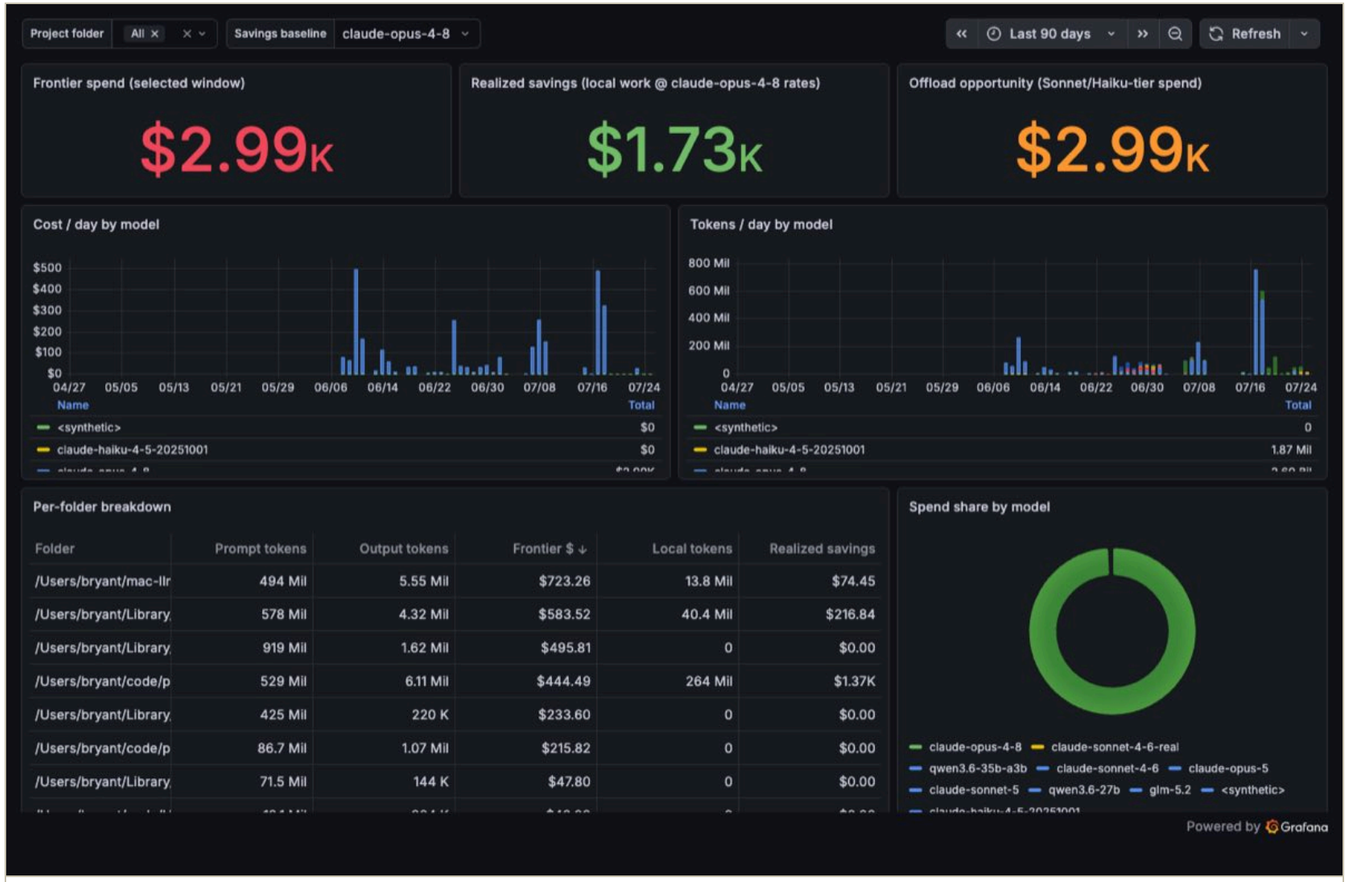
One config change. No model change.

THEN ROUTING DID THE REST

$61

What we actually paid

YOU CAN'T MANAGE WHAT YOU CAN'T SEE.



Live board, **all projects** — filter to plate-solver in the room. Alt-tab and drive it; this is the backup.

THE DISTRIBUTION

Volume sits on the cheap tier; spend concentrates in a handful of expensive calls. You cannot find those without this board.

THE TREND

Realized savings, tracked continuously — the counterfactual as a live number, not something reconstructed after the fact for a slide.

Tokens is an input metric. Cost per unit of delivered work is a business metric. Only one of them belongs on an executive's dashboard.

THE UNIT THAT MATTERS

COST PER OUTCOME, NOT COST PER TOKEN.

A cheaper model is not economical if it needs longer prompts, more retries, or worse answers.

Tokens are an *input* metric. Nobody sets a budget in tokens, and no executive can govern against one. Three questions turn it into something they can:

Task success — did the work actually finish, to a standard you would accept?

Cost-to-serve — what did the *whole* interaction cost, retries included?

Threshold — what accuracy, latency and risk are you willing to trade for it?

THE SAME PROJECT	COMMITS	PAID	EACH
BY HAND 9–11 Jun	24	$405	$16.88
ON THE LOOP 25 Jun – 1 Jul	52	$61.13	$1.18

A commit is not a uniform unit of work — but both weeks are counted the same way, and every commit in the loop week passed an independent judge and the full gate suite before it landed. **14× on the metric an exec can actually govern against.**

TAKE THIS BACK TO YOUR TEAM

THREE MOVES, NONE ABOUT PRICING.

01 ROUTE BY SIZE, NOT BY HABIT

Most tasks don't need the frontier model, and the ones that do are identifiable before you dispatch them. A router is cheaper than a better model.

02 CACHE AGGRESSIVELY

The single biggest lever here, and the one nobody measures. Ask your team what their cache hit rate is. Most cannot answer.

03 BOUND THE CONTEXT

Long-lived agents get expensive because context compounds. Ending the process is a cost control — a fresh agent per task keeps the bill flat as the project grows.

The whole talk in one line: we did not get cheaper by finding cheaper tokens. We got cheaper by changing where the work runs, and how often we pay for the same context.

THREE THINGS TO ARGUE ABOUT.

- WHERE ARE YOU PAYING FRONTIER PRICES FOR WORK A SMALL MODEL COULD DO?
- DO YOU KNOW YOUR CACHE HIT RATE? IT IS USUALLY THE BIGGEST LEVER, AND ALMOST NOBODY MEASURES IT.
- DO YOU KNOW YOUR COST PER DELIVERED THING — OR JUST YOUR MONTHLY BILL?

Backup follows: the two expensive failure modes, and provenance for every number in this deck.

THE FIRST THREE DAYS COST MORE.

PHASE	TOKENS	PAID
JUN 9–11 · BY HAND 19% local	194 M	$405
JUN 25 – JUL 1 · LOOP 85% local	320 M	$61

June 10 alone was $313 — one day, 1,744 calls, five times the entire loop week. That day is what paid for the architecture on slide three.

THE THREE FIXES, ALL IN V1 HISTORY

f207947 Jun 10 — move the judge off the frontier tier, and **trim the orchestrator's context**. The first cost fix was a context fix.

ef871a9 Jun 27 — offline mode with a **deferred judge queue**, so review batches instead of blocking a paid seat.

0af42c4 Jun 29 — a **--judge-model** flag to route review to a separate, cheaper account entirely.

And the loop caught real defects for free. 1885bba · Jun 28 — the judge failed the SV2 task on step ordering, and the loop opened its own follow-up rather than merging it or stalling. That review cost nothing marginal; catching it at frontier prices would not have.

APPENDIX B

WHERE EVERY NUMBER CAME FROM.

CLAIM	VALUE	SOURCE
Scope — ralph-loop window	Jun 25 — Jul 1, 2026	first loop task → 8932c62 workspace integration; 8f32748 post-impl archive
Repo lineage	bryantharpe/plate-solver	tag v1-original = 185128e (absent on tharpeslalom)
Corroborating cutoff	104de07 · Jun 30	tharpeslalom/main tip = merge-base; repo stops where the run stops
Total calls in window	6,922	usage_events, folder = /Users/bryant/code/plate-solver
Tokens delivered	320.0 M	SUM(input + output + cache_creation + cache_read)
Actually paid	$61.13	usage_events × model_prices
Ran on a local model	84.9% of calls	model_prices.is_local
Same workload, all Opus	$573.83	counterfactual at $5/$25/$6.25/$0.50 per MTok
Saved by caching	$1,121.97	cache_read_tokens × (input rate − cache-read rate)
Frontier calls / spend	381 · $60.80	claude-opus-4-8 — human monitoring
Judge model	glm-5.2 · 654 calls	routed via grind --judge-model (0af42c4)
Local author model	qwen3.6-27b	litellm/config.yaml — sonnet alias remapped to local
Manual phase	$405.05 · 194.0 M	Jun 9–11, same folder, 18.7% local
Unit cost	$2.09 → $0.19 /MTok	paid ÷ tokens, per phase
Solve time · stars matched	1.8 ms · 47	ps-web UI screenshot, docs/screenshots/

Not counted: work after Jul 1 moved to a home DGX under a different orchestrator and is not in this database. Every figure above is re-runnable live against the Postgres store behind the Grafana board.